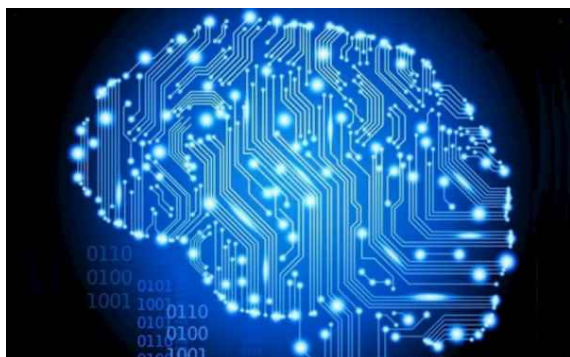


```
print("modifier_ob",modifier_ob)
# put mirror modifier on modifier_ob
mirror_mod = modifier_ob.modifiers.new("mirror_mirror","MIRROR")
# set mirror object to mirror_ob
mirror_mod.mirror_object = mirror_ob
```

《计算机科学导论》课程

算法 · 贪心算法

上海交通大学计算机科学导论课程组



致谢：本课件由廖铭鼎@CS2017M.SJTU编写初稿，陈奕廷@CS2021D.SJTU修缮完善，张嘉乐@CS2022D.SJTU优化，高晓沅终校定稿。

目 录

01

贪心算法简介

02

最短路算法

03

贪心算法应用

贪心算法（greedy algorithm），又称**贪婪算法**，是一种在每一步选择中都采取在当前状态下最好或最优（即最有利）的选择，从而希望导致结果是最好或最优的算法。

为什么学贪心算法？

- 解现实问题快：导航、物流调度、网络路由、压缩编码（哈夫曼）、资源分配等广泛可用。
- 思想可迁移：局部最优-全局最优的张力，帮助快速做决定与构建启发式。
- 认识边界：学会判断“何时成立、何时失效”，形成算法素养与建模能力。

硬币找零问题

- 给定货币面额：1美分(penny)、5美分(nickel)、10美分(dime)、25美分(quarter)、1美元(=100美分)，设计一种使用最少数量硬币来找零的方法。
- 贪心算法：对硬币面额倒序排列，选择最大面额的硬币找零，直到余额少于该面值，换下一面额的硬币。
- 例：需要支付2.87美元



1美元
dollar



25美分
quarter



10美分
dime



5美分
nickel



1美分
penny

贪心算法的“三问法”

- 局部选择：此问题有“看起来合理的局部最优”选择标准吗？（如最早结束时间、性价比、最大面值）
- 最优子结构/可交换性：问题被拆成若干阶段或子问题，每做出一个选择后，剩下的仍然是同一类问题。是否能用“交换论证/替换法”证明本地选择不劣于最优解中的对应选择？
- 反例检验：能否快速构造反例来推翻一个贪心标准？若能，是否还有受限版本成立（如规范货币、非负边、可切分物品）？

硬币找零问题

- 给定货币面额：1美分(penny)、5美分(nickel)、10美分(dime)、25美分(quarter)、1美元(=100美分)，设计一种使用最少数量硬币来找零的方法。
- 贪心算法：对硬币面额倒序排列，选择最大面额的硬币找零，直到余额少于该面值，换下一面额的硬币。
- 例：需要支付2.87美元



1美元
dollar

25美分
quarter

10美分
dime

5美分
nickel

1美分
penny

$$287 - 100 - 100 - 25 - 25 - 25 - 10 - 1 - 1 = 0$$

贪心算法正确吗？

考虑美国邮票系统，有：1美分、10美分、21美分、34美分、70美分、100美分、350美分、1225美分、1500美分共9种面值邮票



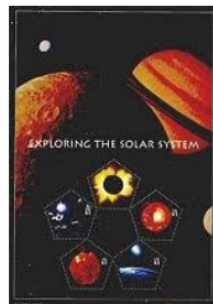
贪心算法正确吗？

反例：支付140美分时：

- 贪心算法结果： $140 = 100 + 34 + 1 + 1 + 1 + 1 + 1 + 1$ ，共需要8张邮票
- 最优结果： $140 = 70 + 70$ ，需要2张邮票

在硬币找零问题中，贪心算法不一定是完全正确的

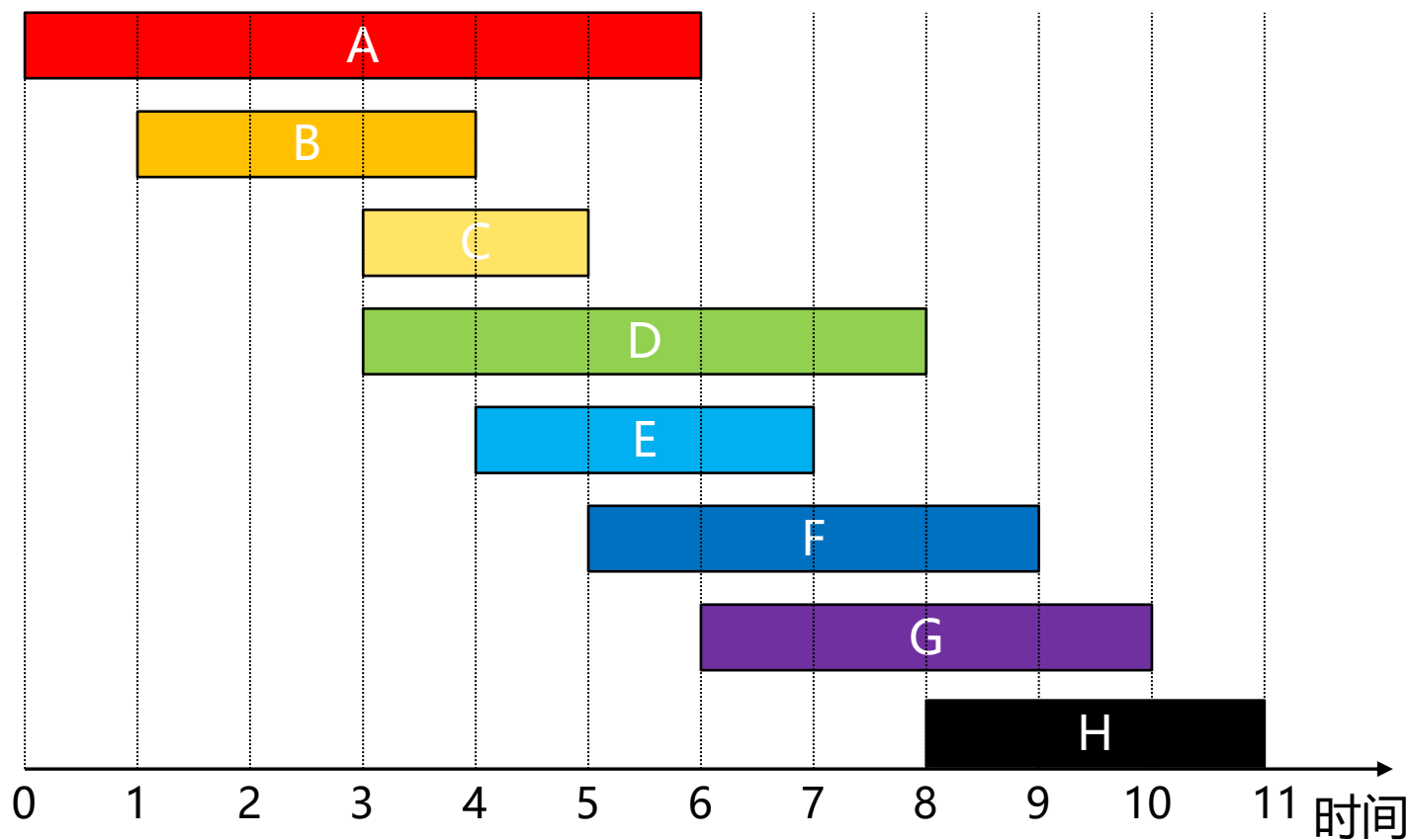
贪心算法正确性与
问题设定有关



智能制造中的生产调度问题

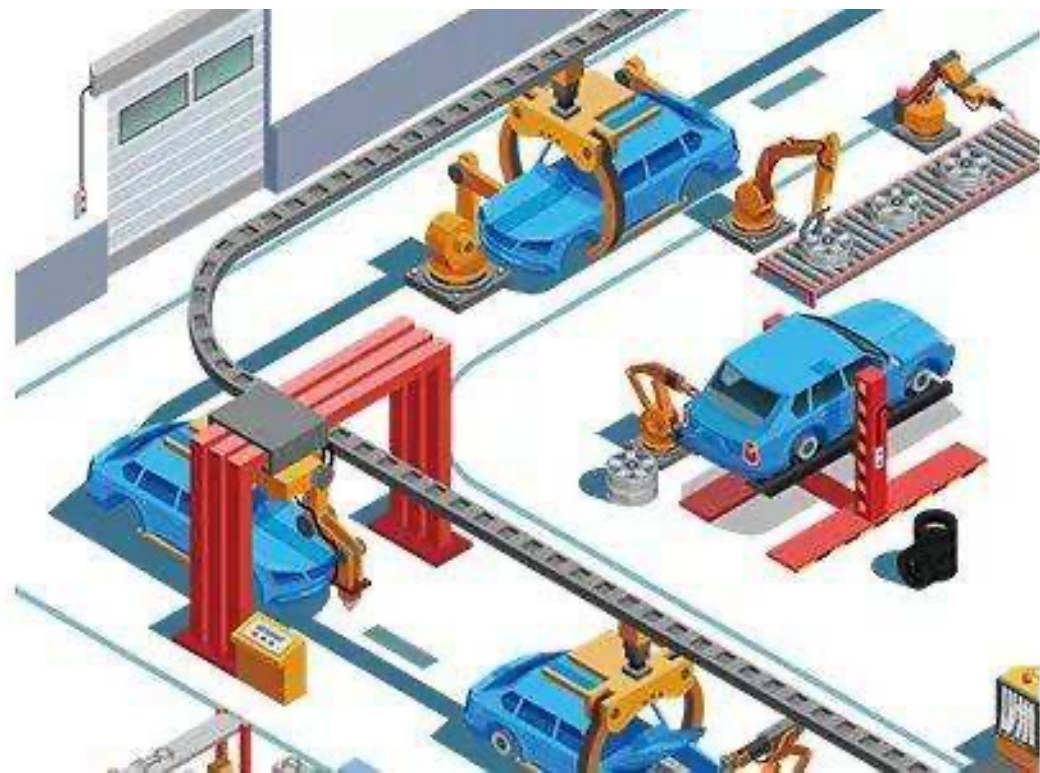
交互提问，策略？

- 在一个工厂中，给定一系列商品，每个商品有拟定的开始时间与结束时间。
- 工厂流水线同一时间只能生产一件商品，请问在一段时间内最多可以生产几件商品？



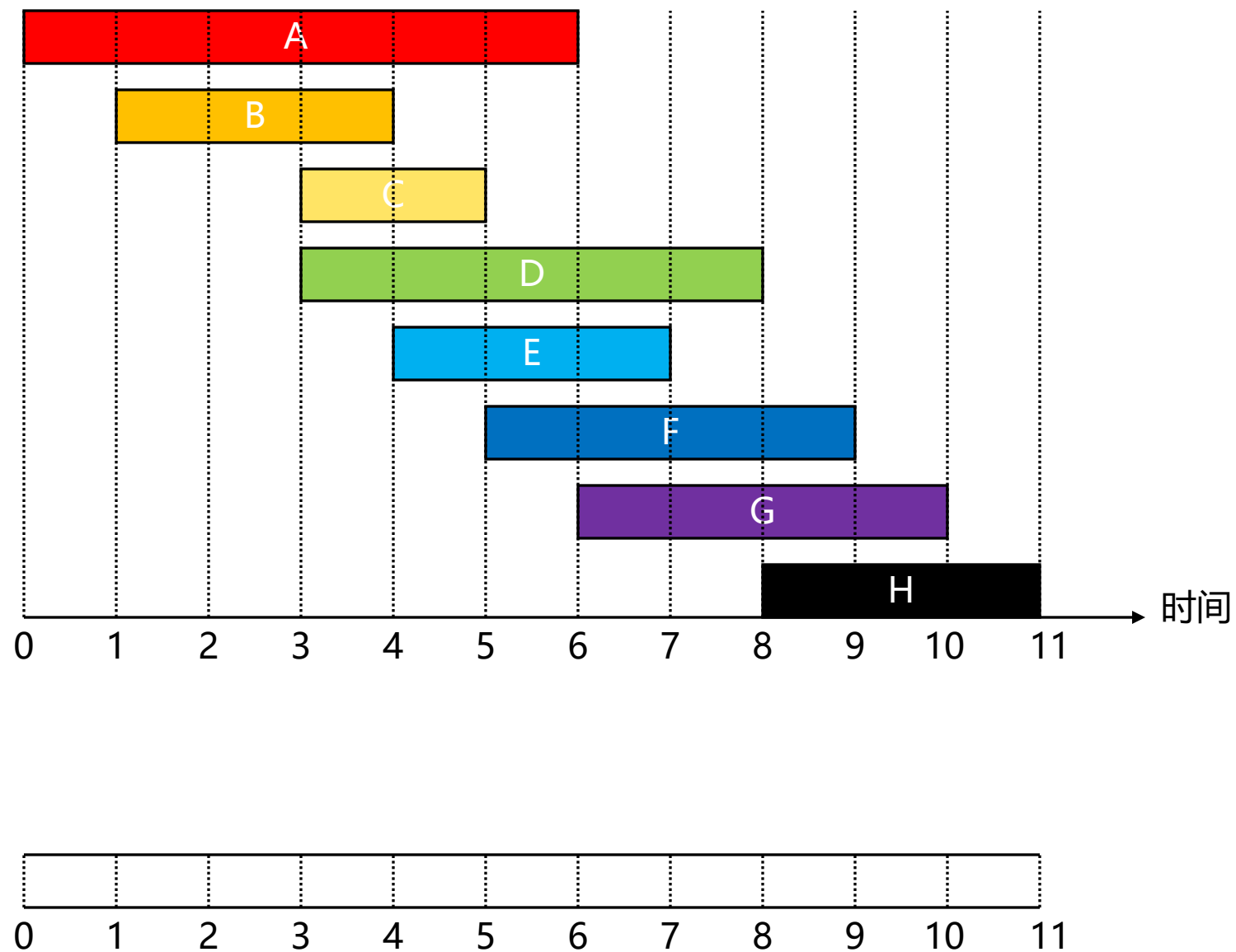
2025/10/19

上海交通大学-计算机科学导论-贪心算法

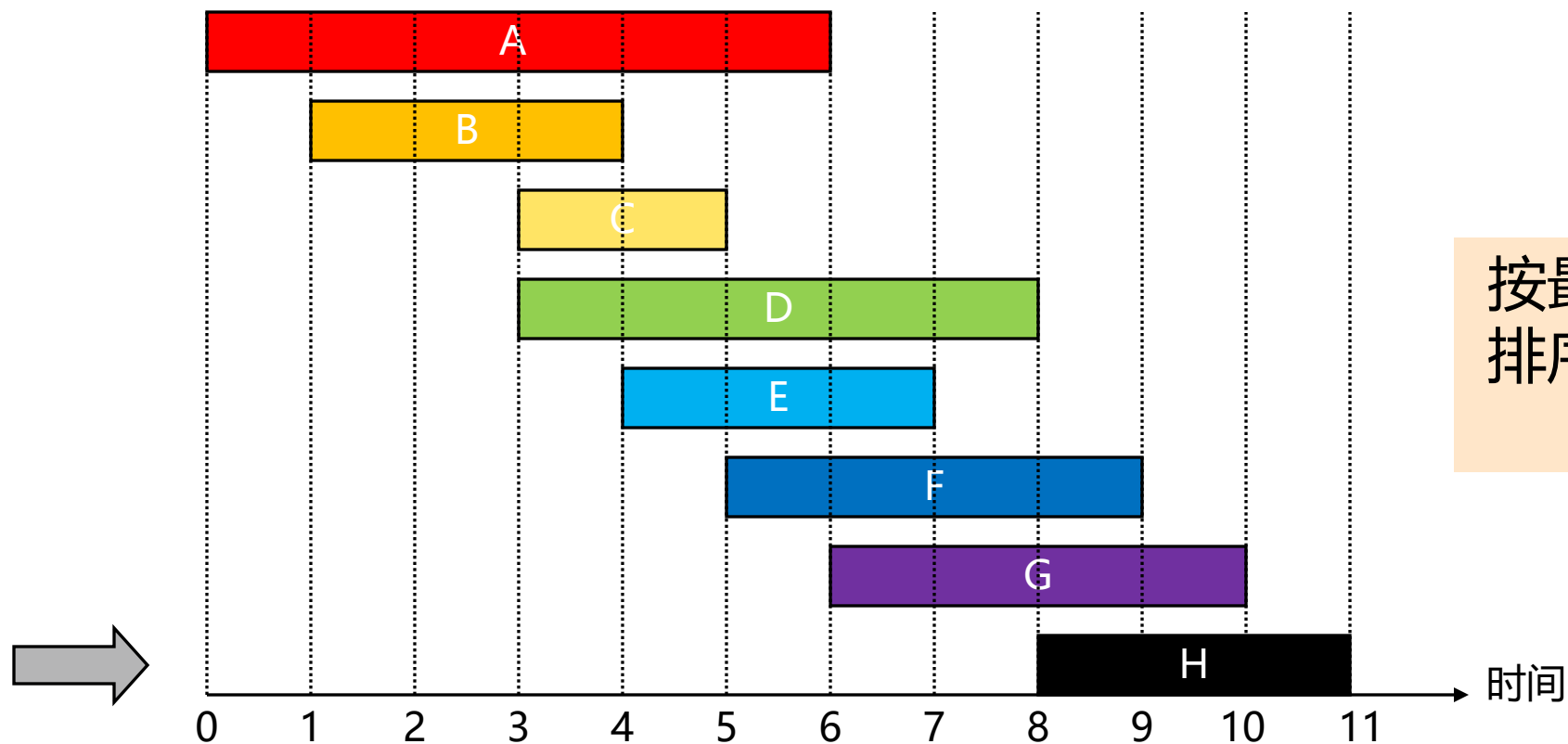


汽车智能制造生产线

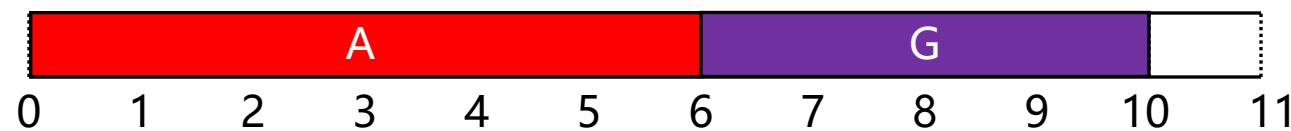
求解尝试：按最早开始时间排序



求解尝试：按最早开始时间排序

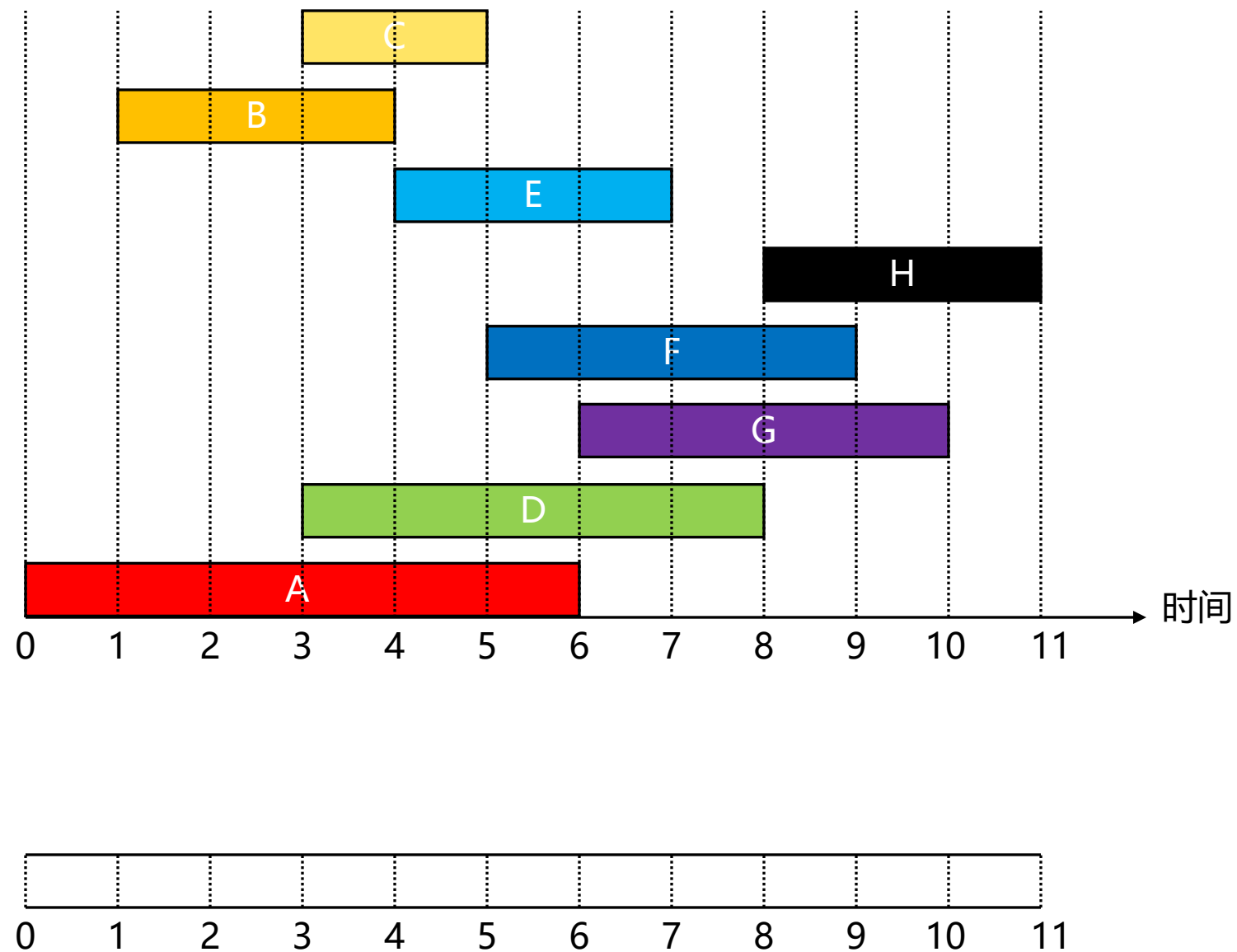


按最早开始时间
排序不能得到最
优策略!

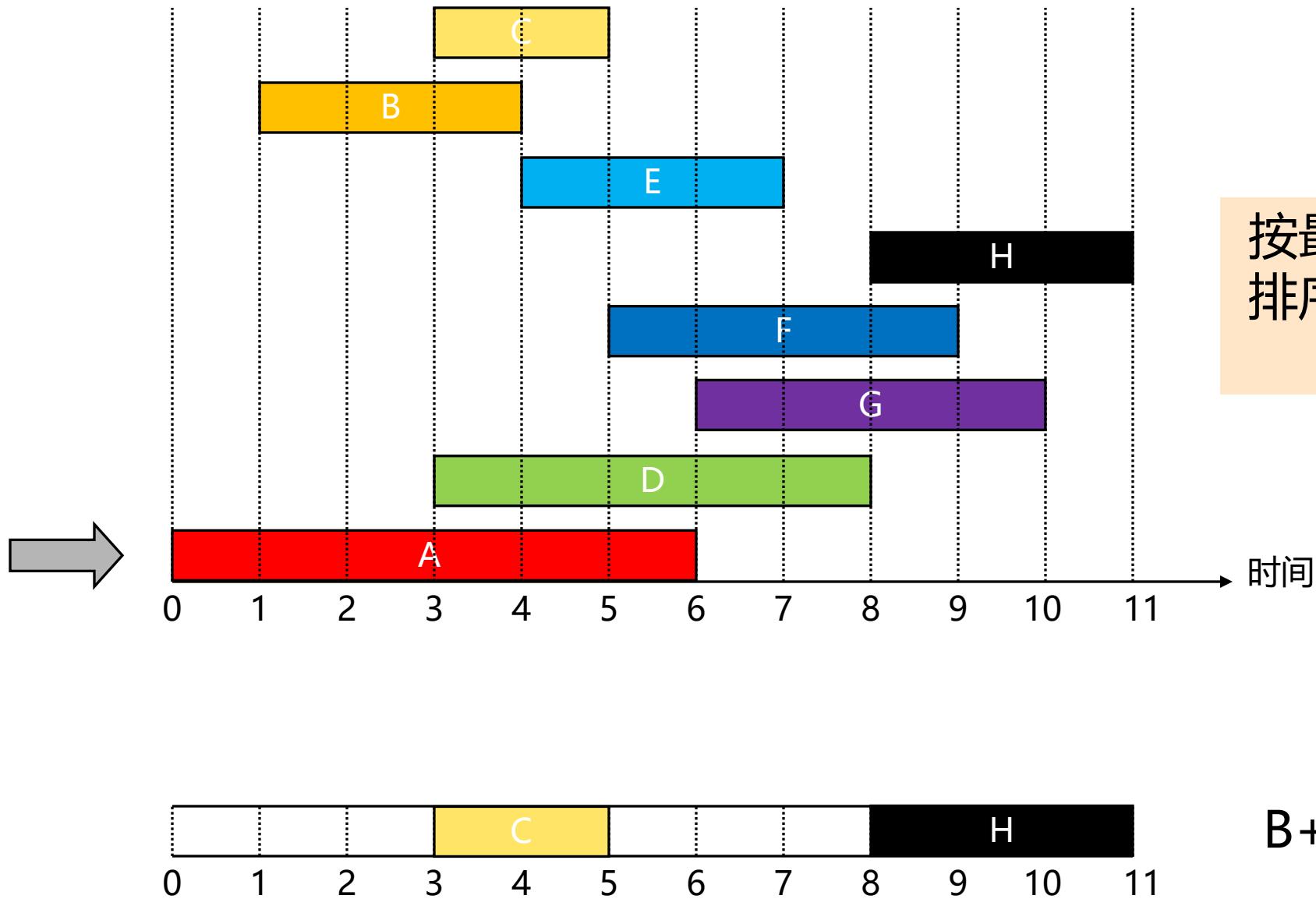


$$B+E+H > A+G$$

求解尝试：按最短完成时间排序



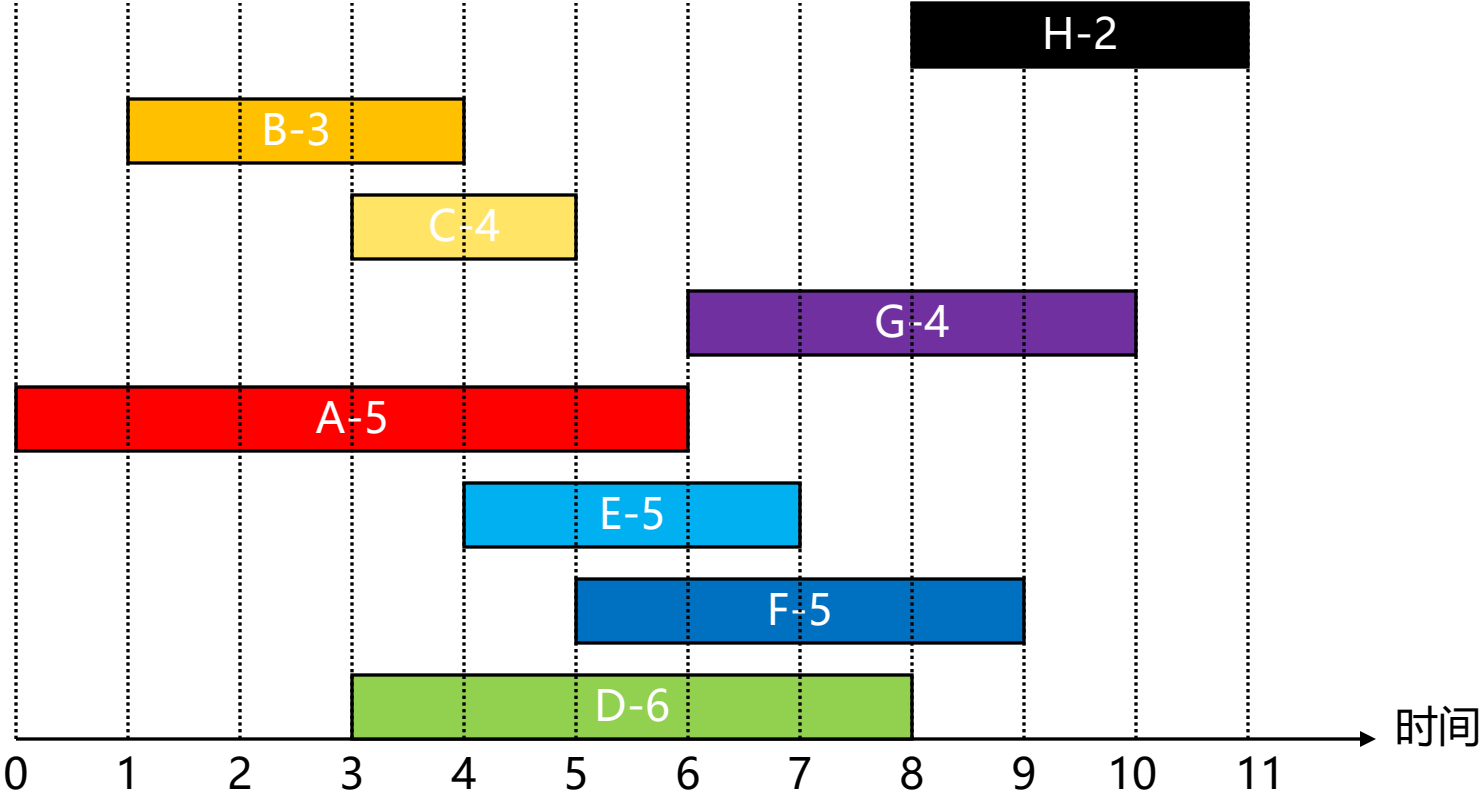
求解尝试：按最短完成时间排序



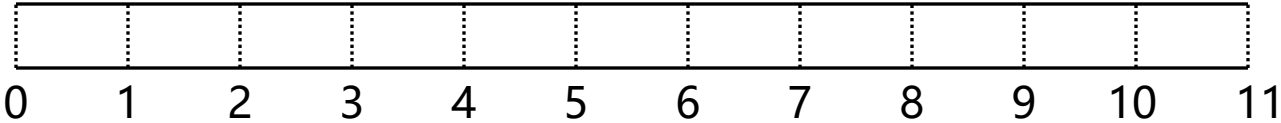
按最短完成时间
排序不能得到最
优策略！

$$B + E + H > C + H$$

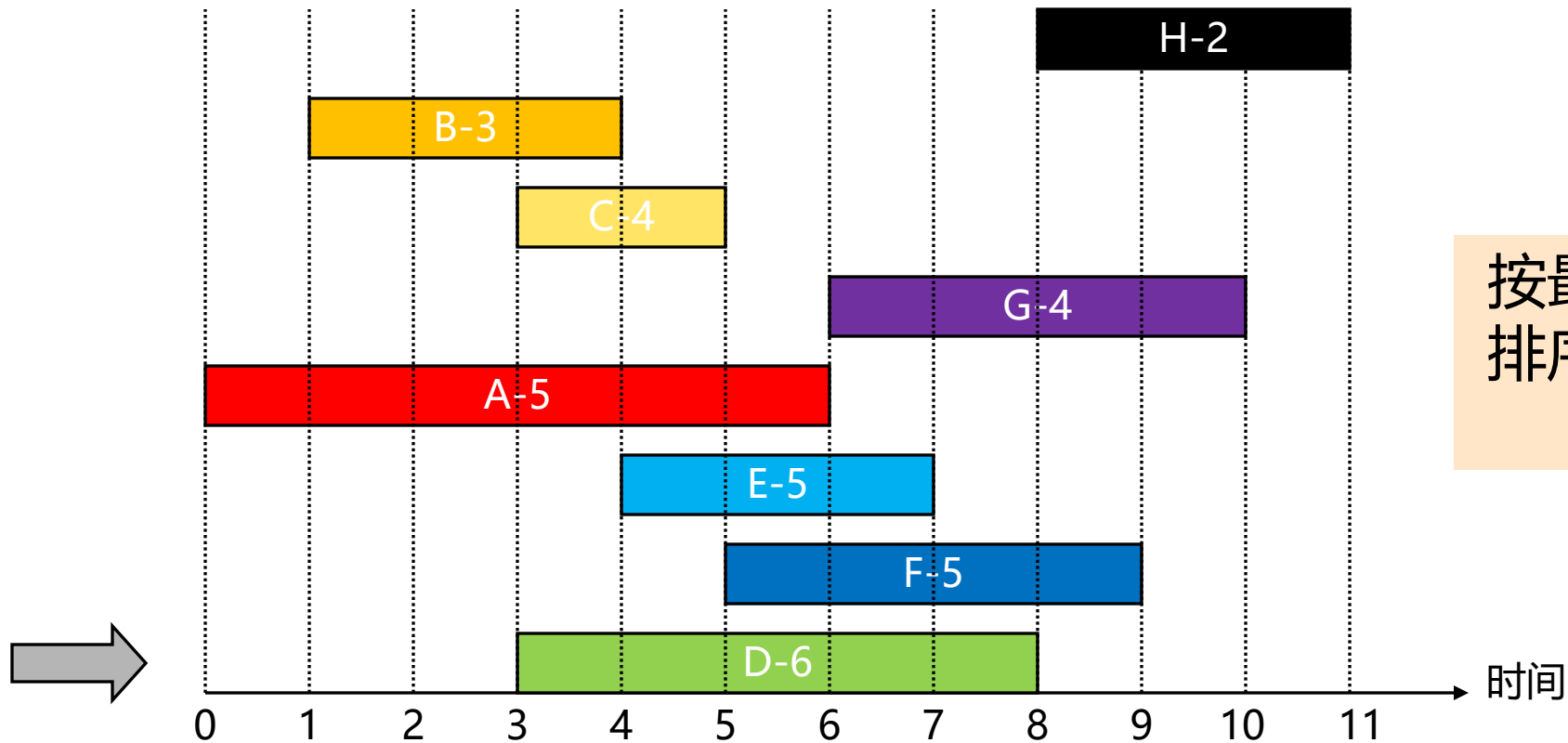
求解尝试：按最少冲突优先排序



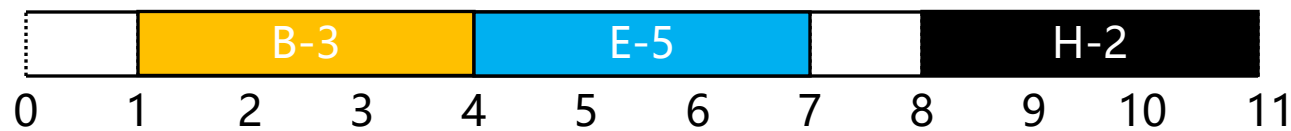
(任务内的数字表示冲突任务数量)



求解尝试：按最少冲突优先排序

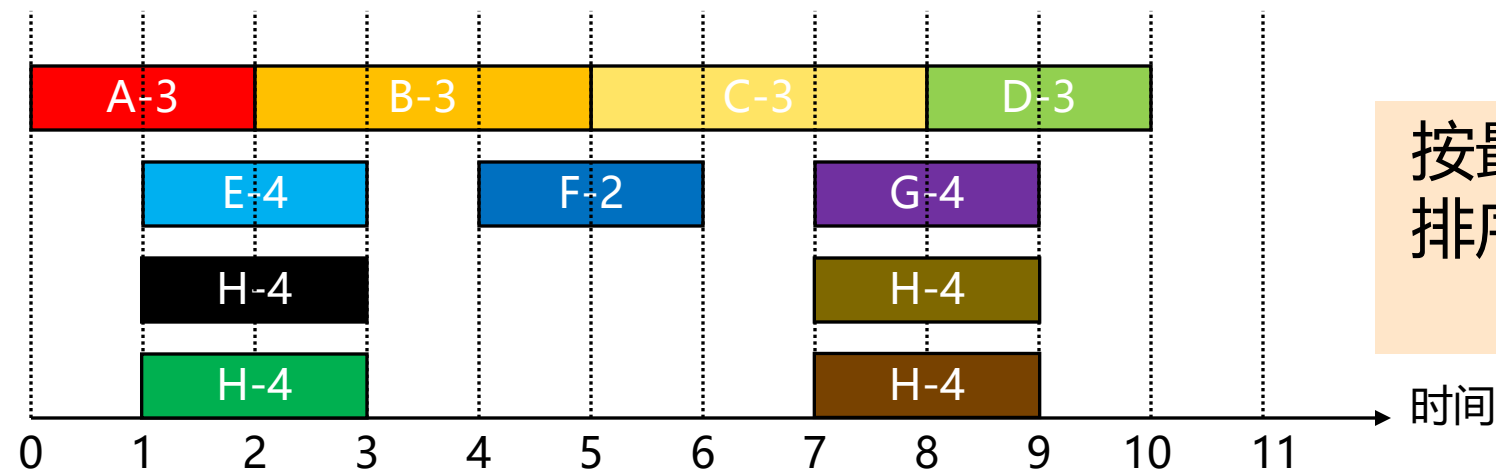


(任务内的数字表示冲突任务数量)



求解尝试：按最少冲突优先排序

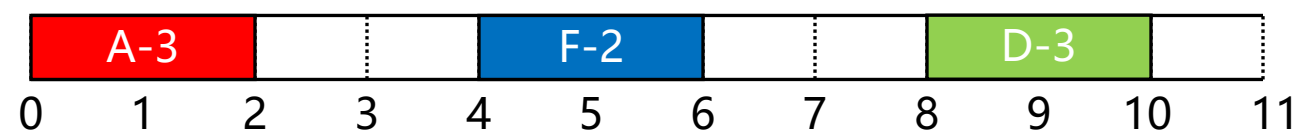
如果商品生产时间分布如下所示.....



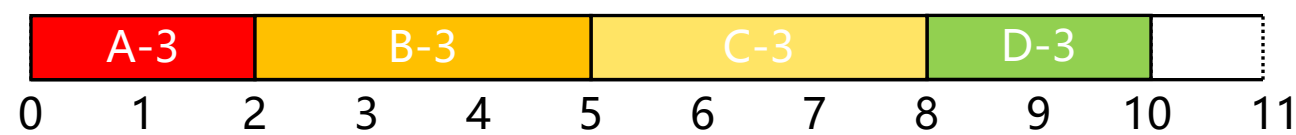
按最少冲突优先
排序不能得到最
优策略！

(任务内的数字表示冲突任务数量)

最少冲突优先方案

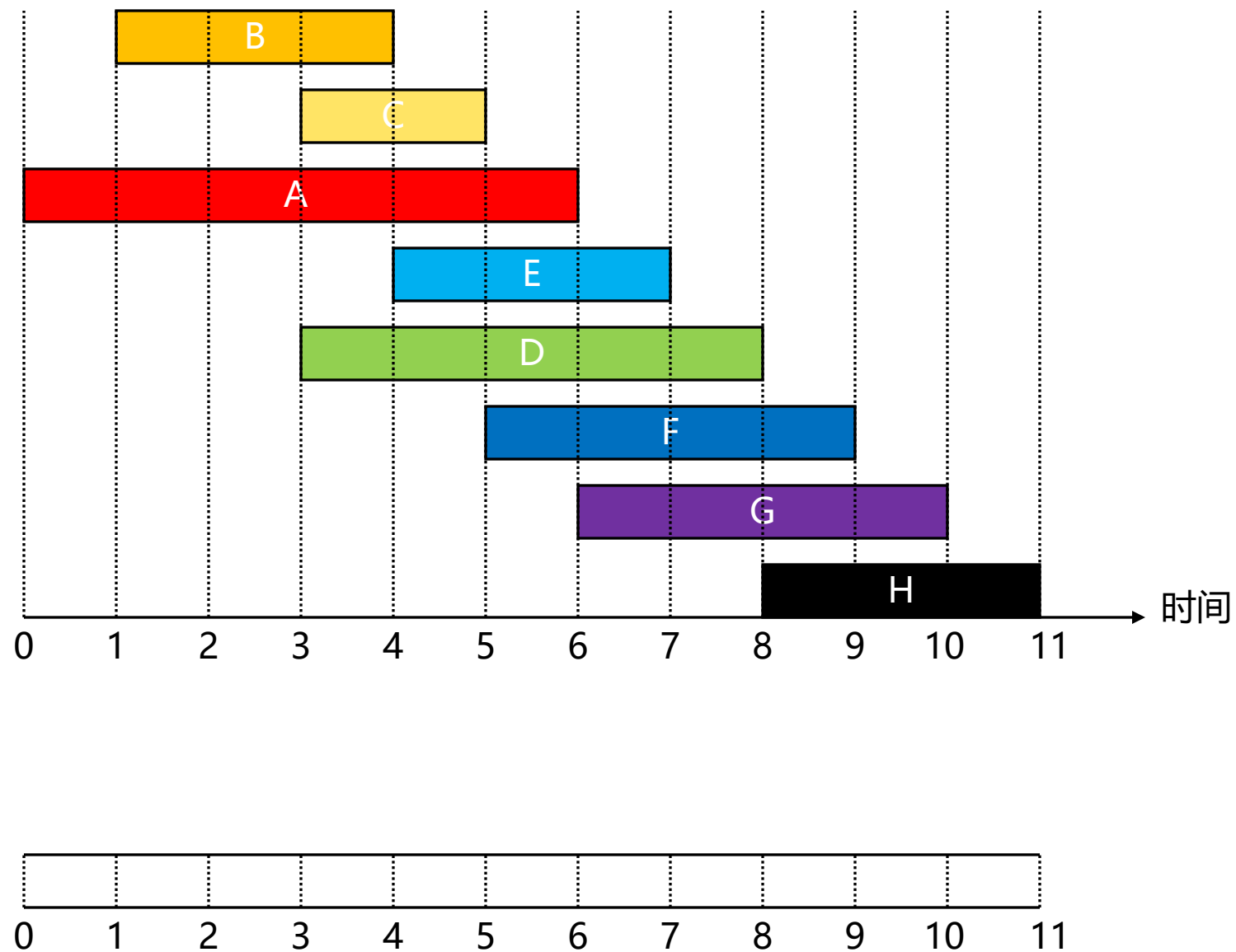


最优方案

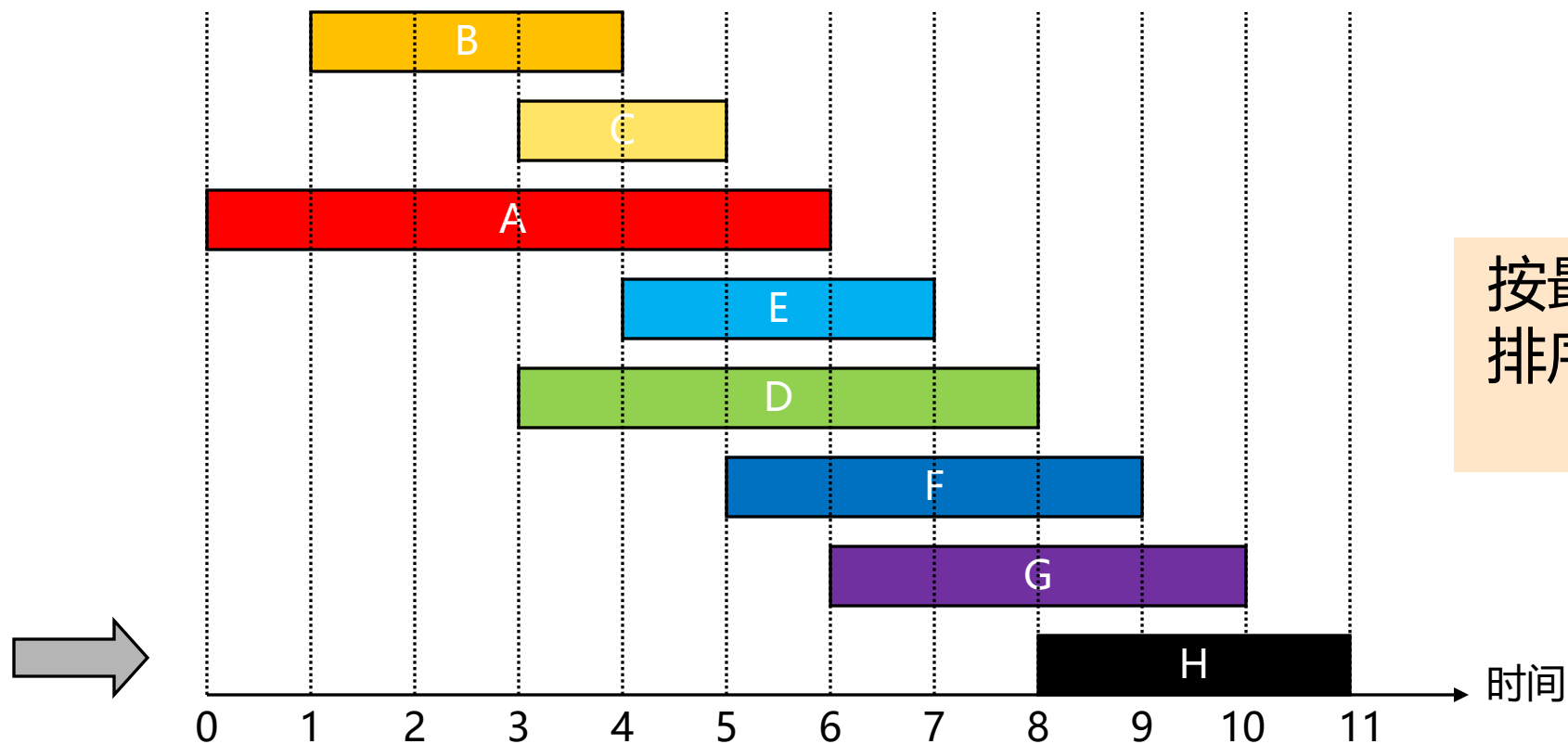


判断贪心策略能
否确保得到最优
解十分关键

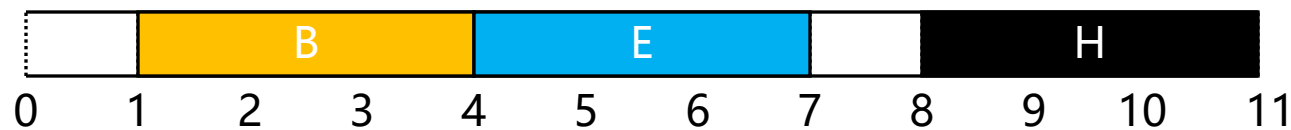
求解尝试：按最早结束时间排序



求解尝试：按最早结束时间排序



按最早结束时间
排序可以得到最
优策略!



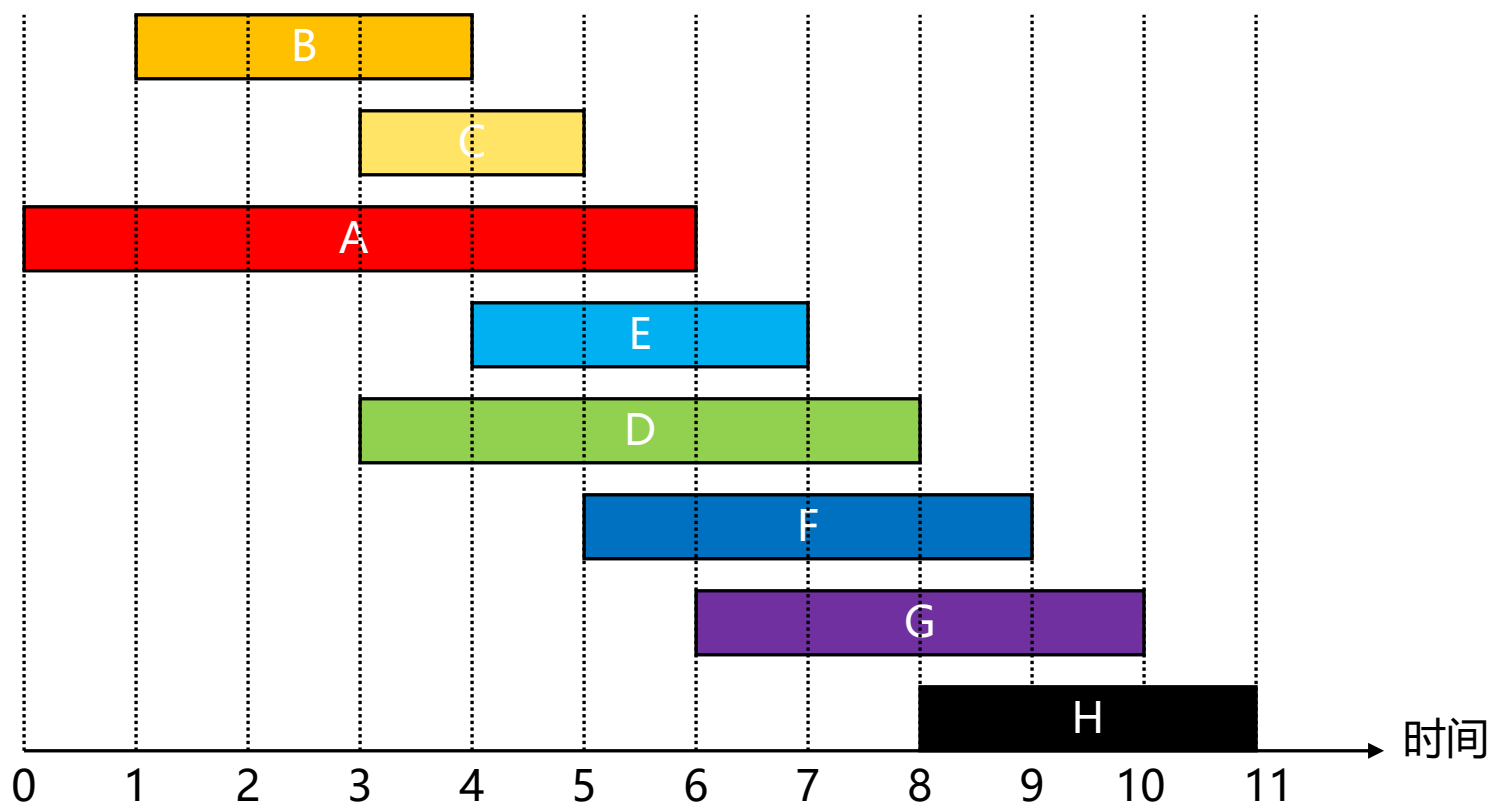
求解策略比较分析

- **最早开始时间**: 用开始时间的升序排列逐一考虑生产商品
- **最少冲突优先**: 计算与商品冲突的数量, 升序排列逐一考虑生产商品
- **最短执行时间**: 用执行时间(开始-结束)的升序排列逐一考虑生产商品
- **最早结束时间**: 用结束时间的升序排列逐一考虑生产商品

最早结束时间可以得到最优解!



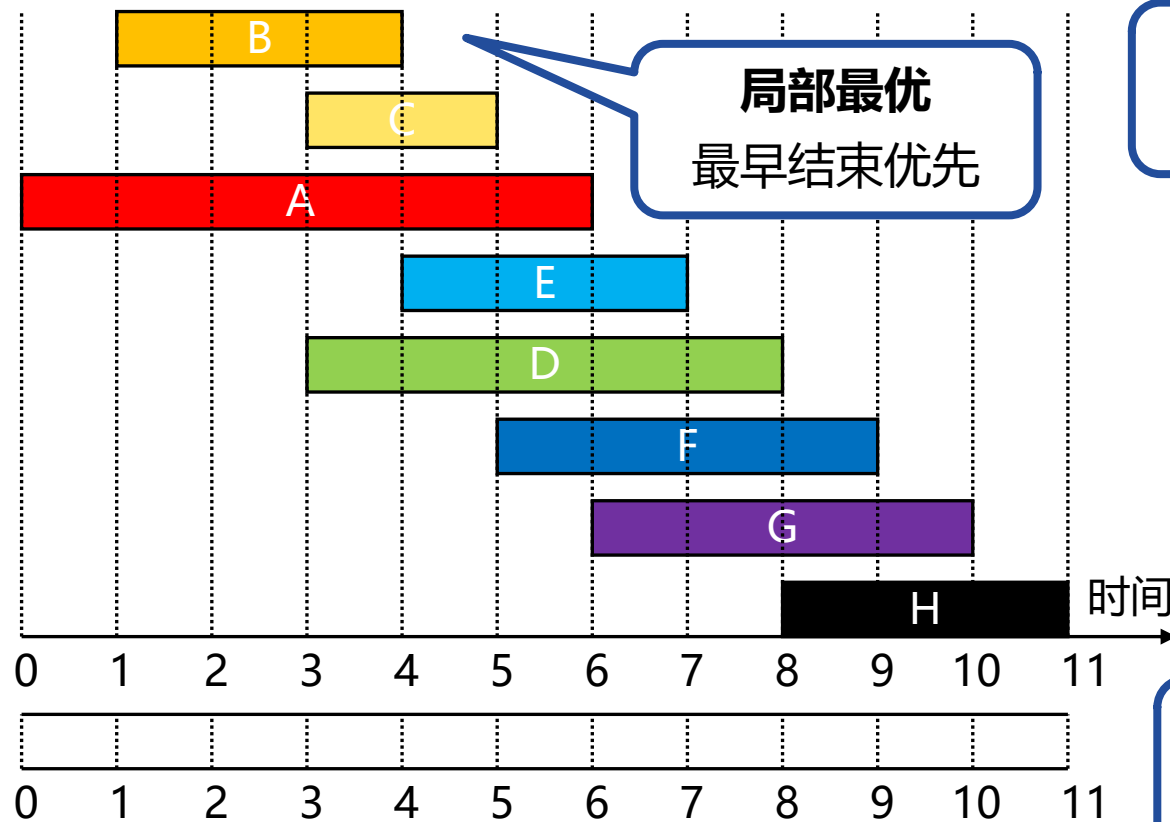
怎样设置求解策略?



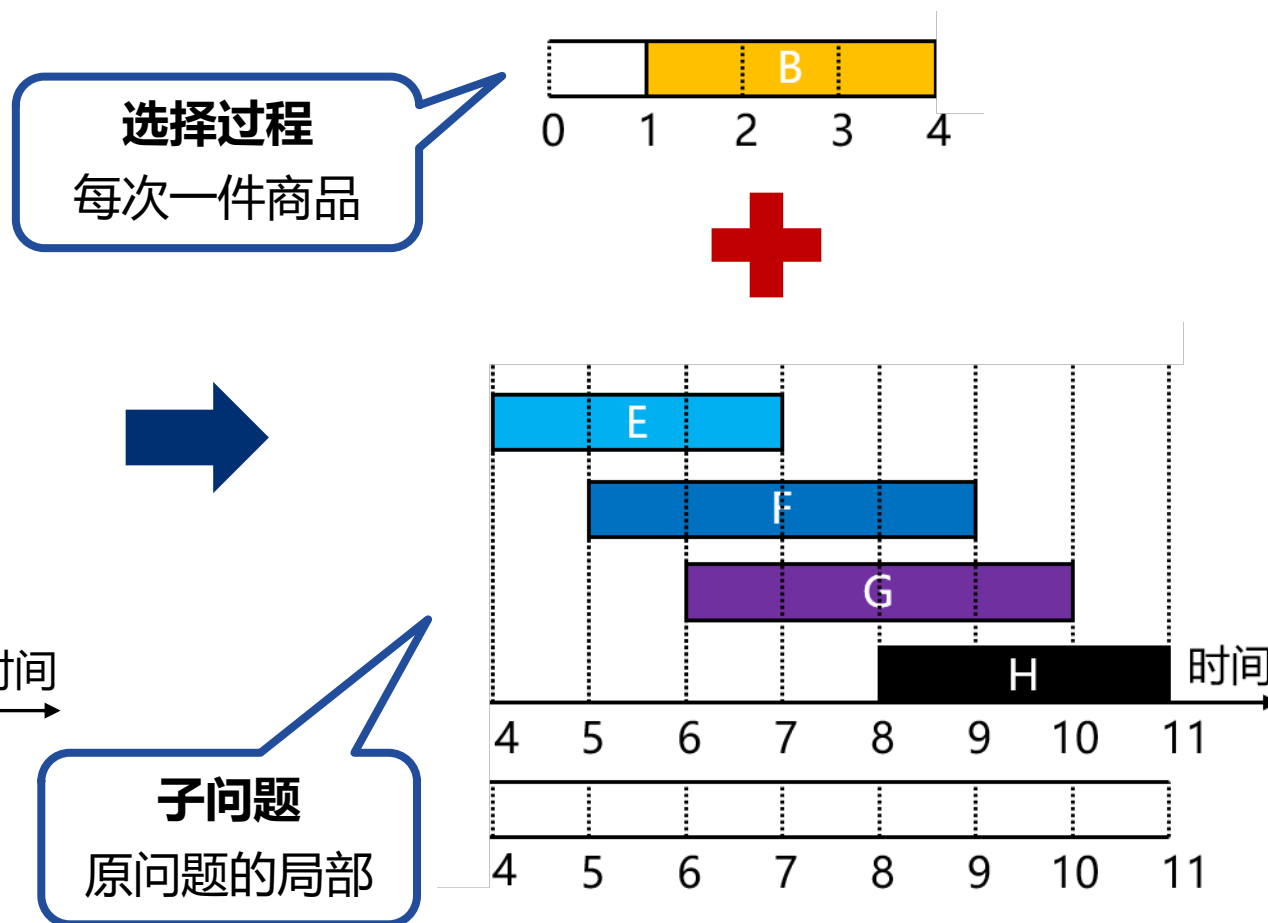
生产调度问题求解剖析

生产调度问题的关键点

- 判断贪心策略能否得到最优解



- 问题维度增加可能导致贪心策略失效



生产调度算法的一般形式

第一步：对目标对象进行拆解，分为阶段性求解

第二步：设计合理策略，逐一考虑每个对象的可行性，进行选择

第三步：将每步得到的结果合并为最终结果

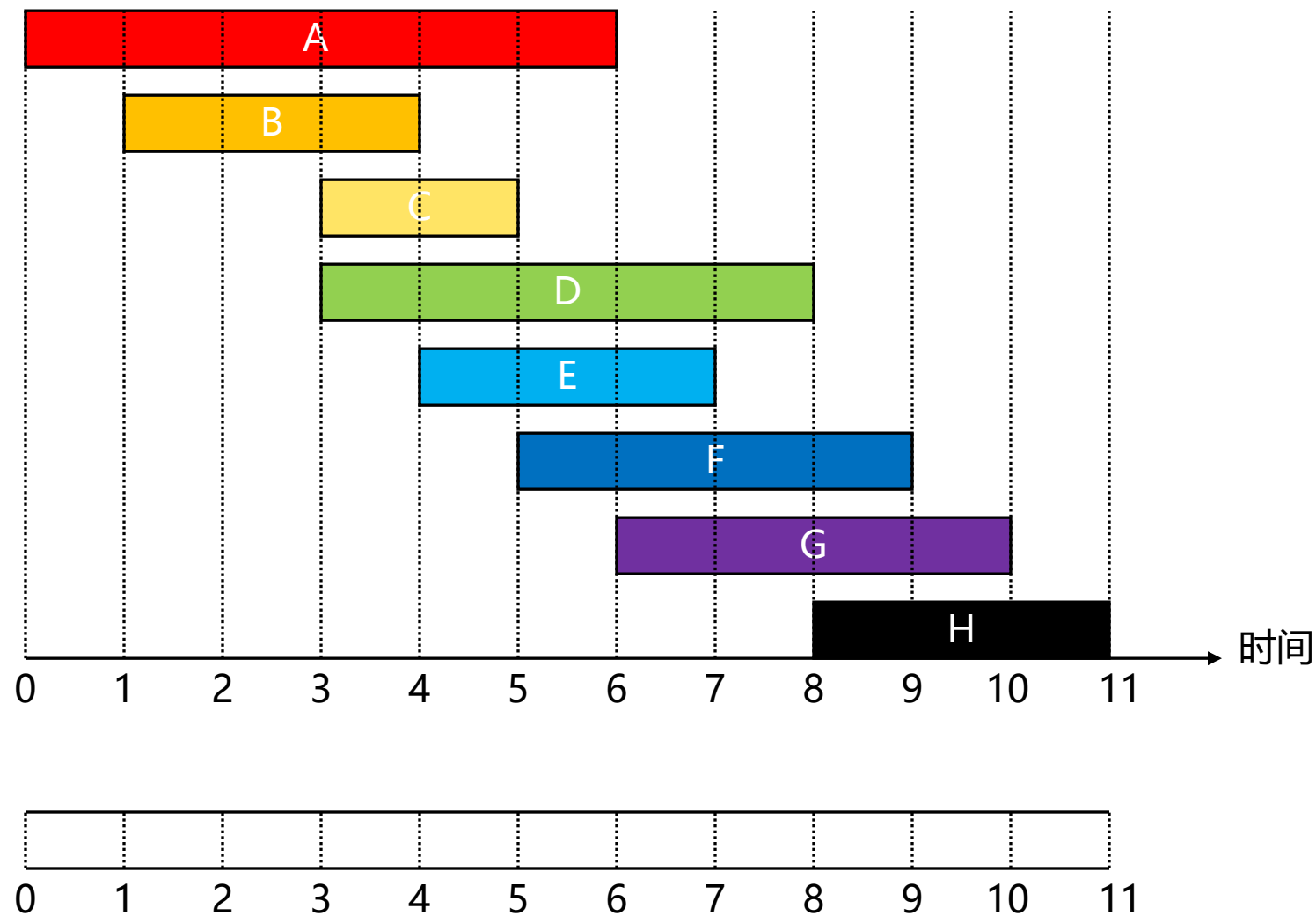
商品生产算法（伪代码）

```
1  将商品按结束时间升序排序，重新编号为 $f_1 \leq f_2 \leq \dots \leq f_n$ ;  
2  令集合 $A = \emptyset$ ;  
3  for  $j = 1$  to  $n$   
4      if 第 $j$ 个商品与 $A$ 中已选商品的生产时间不冲突  
5           $A = A \cup \{j\}$ ;  
6      endif  
7  endfor  
8  返回商品集合 $A$ 
```

“贪心”

每次只考虑当前最优情况

工厂生产问题：贪心算法的解唯一吗？



工厂生产问题：按最早结束时间排序的解不唯一

贪心算法结果 A_g ：

- {B, E, H}

最优解 A^* （可能不止一个，本例中仅有一个）：

- {B, E, H}

可行解集合（可行、但不一定最优）：

- $\emptyset, \{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}, \{G\}, \{H\};$
- $\{A, G\}, \{A, H\}, \{B, E\}, \{B, F\}, \{B, G\}, \{B, H\}, \{C, F\}, \{C, G\}, \{C, H\}, \{D, H\}, \{E, H\};$
- {B, E, H}

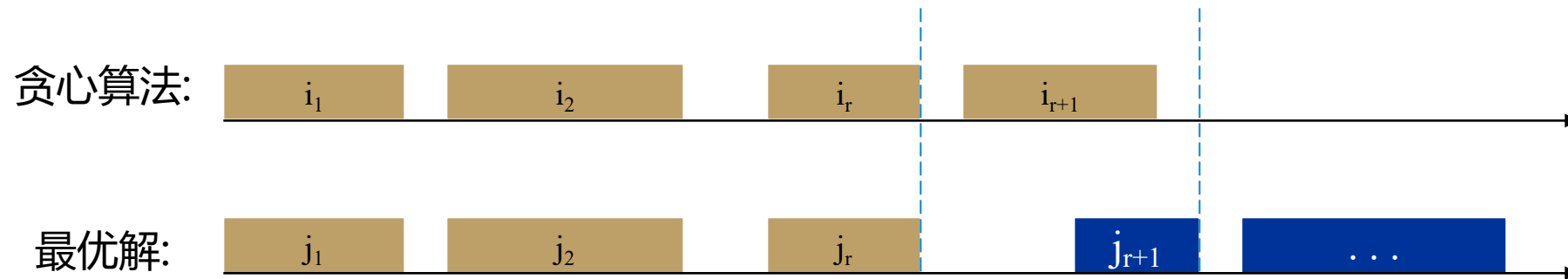
工厂生产问题：按最早结束时间排序的正确性证明

假设 $A_g = \{i_1, \dots, i_{|A_g|}\}$ 是贪心算法结果， $A^* = \{j_1, \dots, j_{|A^*|}\}$ 是工厂生产问题的最优解，将 A^* 中的商品按结束时间排序，设其前 r 个商品均与贪心算法结果一致，且 A^* 是所有最优解中 r 最大的一个。

若 $r = |A_g|$ ，则 $A_g = A^*$ ，贪心算法结果即为最优解。

若 $r \neq |A_g|$ ，此时 j_{r+1} 为 A^* 中第一个不符合贪心策略的商品，根据贪心算法策略， i_{r+1} 是在第 r 步之后所有商品中结束时间最早的，则将 A^* 中的 j_{r+1} 替换为 i_{r+1} 并不影响结果的最优性，即 $A^* \setminus \{j_{r+1}\} \cup \{i_{r+1}\}$ 仍是最优解，但此时它与 A_g 的前 $r + 1$ 个商品均一致，这与 A^* 是所有最优解中 r 最大的一个矛盾，故 $r = |A_g|$ 。

该证明也可用数学归纳法的方式进行替换，直至 $r = |A_g|$ 。



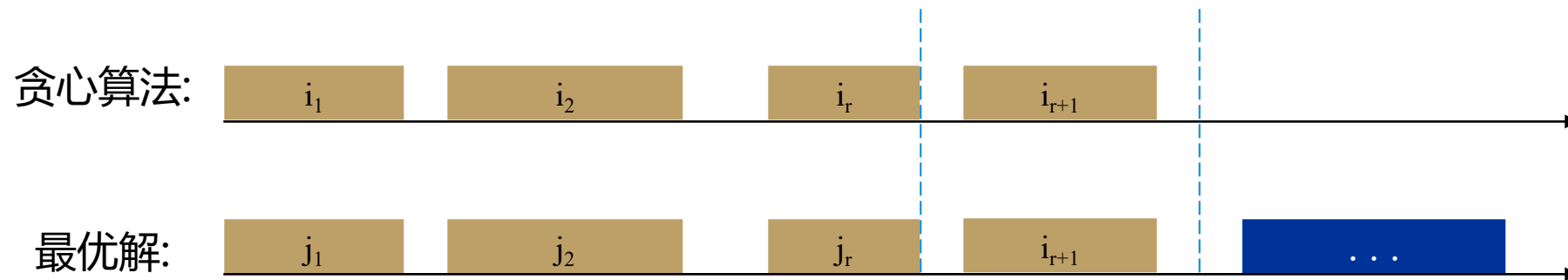
工厂生产问题：按最早结束时间排序的正确性证明

假设 $A_g = \{i_1, \dots, i_{|A_g|}\}$ 是贪心算法结果， $A^* = \{j_1, \dots, j_{|A^*|}\}$ 是工厂生产问题的最优解，将 A^* 中的商品按结束时间排序，设其前 r 个商品均与贪心算法结果一致，且 A^* 是所有最优解中 r 最大的一个。

若 $r = |A_g|$ ，则 $A_g = A^*$ ，贪心算法结果即为最优解。

若 $r \neq |A_g|$ ，此时 j_{r+1} 为 A^* 中第一个不符合贪心策略的商品，根据贪心算法策略， i_{r+1} 是在第 r 步之后所有商品中结束时间最早的，则将 A^* 中的 j_{r+1} 替换为 i_{r+1} 并不影响结果的最优性，即 $A^* \setminus \{j_{r+1}\} \cup \{i_{r+1}\}$ 仍是最优解，但此时它与 A_g 的前 $r + 1$ 个商品均一致，这与 A^* 是所有最优解中 r 最大的一个矛盾，故 $r = |A_g|$ 。

该证明也可用数学归纳法的方式进行替换，直至 $r = |A_g|$ 。



《一千零一夜》· 阿里巴巴与四十大盗



当阿里巴巴的来到强盗的洞穴里，发现自己只有一个口袋，他应该如何带走的物品呢？

——出自《天方夜谭》民间故事
“Tales from the thousand and one nights”

阿拉伯和波斯，傻傻分不清楚

- 阿拉伯数字不是阿拉伯人发明的，而是波斯人发明的
- 阿里巴巴与四十大盗的故事中，阿里巴巴的设定是波斯人，而不是我们直观上以为的阿拉伯人

背包问题

已知阿里巴巴的口袋只能装下重量不超过 W 的宝物，在洞穴中有 n 件宝物，第 i 件宝物的重量为 w_i 、价值为 v_i ，如何用口袋装下价值最高的宝物？

抽象为**背包问题 (Knapsack Problem)**：有 n 件物品和容量为 W 的背包，第 i 件物品的重量为 w_i 、价值为 v_i ，求问将哪些物品装入背包可使价值总和最大。

更多背包问题

- 完全背包问题：每一件物品可以取无数次；
- 多重背包问题：第 i 件物品可以取 n_i 次
- 混合背包问题：第 i 件物品可以取 1 次、 n_i 次或无数次
- 多维背包问题：每一件物品除了重量外还有其他的“代价”
- ...

背包问题的贪心算法

- 假设阿里巴巴的口袋能装 $10kg$ 重量的宝物，洞穴里有4件宝物：



国王的古董

重量5、价值100
性价比20



皇后的妆盒

重量2、价值40
性价比20



教皇的皇冠

重量3、价值51
性价比17



公主的项链

重量2、价值35
性价比17.5

- 口袋剩余容量： $10-2-5-2=1$

- 宝物总价值： $40+100+35=175$



- 1. 计算物品的性价比 v_i/w_i ;
- 2. 将物品按性价比从高往低排序;
- 3. 将剩余物品中性价比最高的放进背包;
- 4. 重复操作3，直到口袋装满。

让我们再回到背包问题

假设阿里巴巴的口袋能装 $10kg$ 重量的宝物，洞穴里有4件宝物



国王的古董
重量5、价值100
性价比20



皇后的妆盒
重量2、价值40
性价比20



教皇的皇冠
重量3、价值51
性价比17



公主的项链
重量2、价值35
性价比17.5

贪心算法
175

最优结果
191

在背包问题中，贪心算法不一定是对的，但在工程中是快速算法（可以优化为 $(1+\epsilon)$ -近似算法）

贪心算法正确性与问题本质有关

可以利用动态规划（Dynamic Programming）求最优解

目 录

01

贪心算法简介

02

最短路算法

03

贪心算法应用

如何找到最“短”路径？

上海虹桥国际机场 → 上海浦东国际机场 推荐路线（多条）



①路程最短

②时间最快

③费用最低

最短路径问题建模

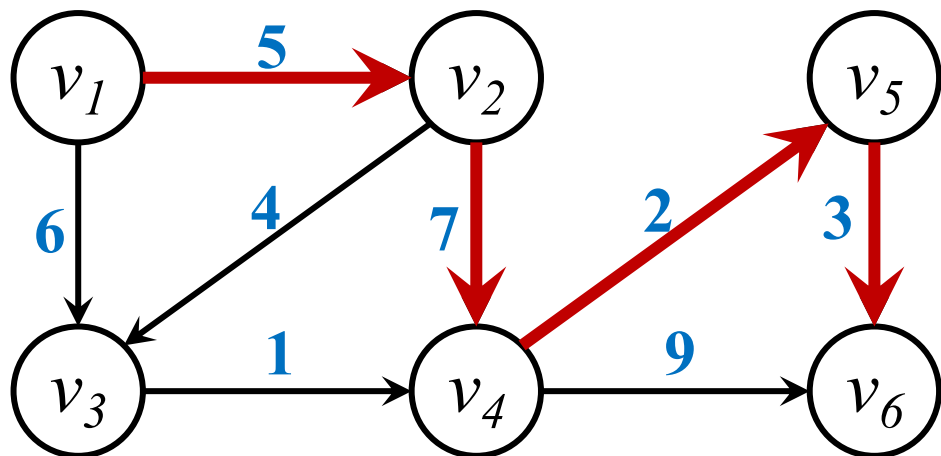
建模: 构造有向图 $G = (V, E)$, V 为顶点集合, E 为有向边集合

权值: 每条边 (u, v) 的权重为 $w(u, v) \geq 0$ (非负权!)

路径: $P = v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_n$

长度: $w(P) = \sum_{i=1}^{n-1} w(v_i, v_{i+1})$

- 道路长度
- 行驶时间
- 通行成本
-



$$w(P(v_1 \rightarrow v_6)) = 17$$

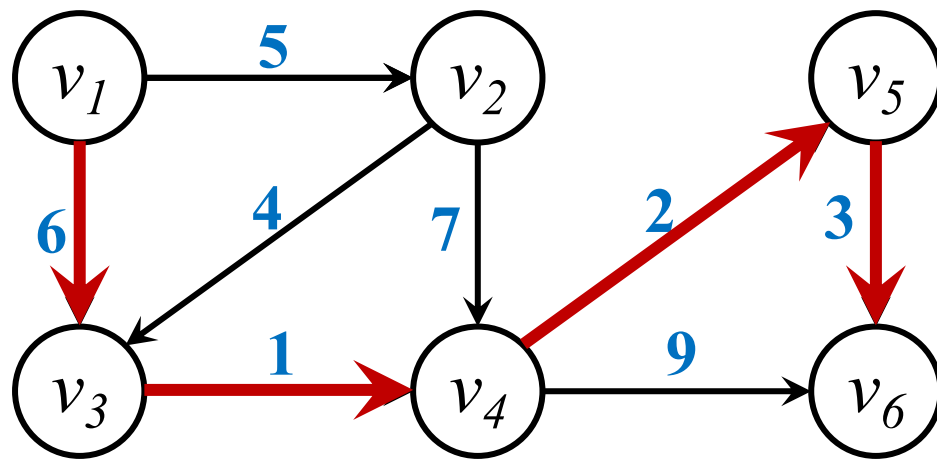
v_1 到 v_6 是否还有更短的路径?

最短路径问题定义

最短路径问题： 给定源点 s ，计算由源点到其他各顶点的最短路径

令 $d(u, v)$ 为从 u 到 v 的最短路径长度，则

$$d(u, v) = \min\{w(P) : P \text{ 是从 } u \text{ 到 } v \text{ 的路}\}$$

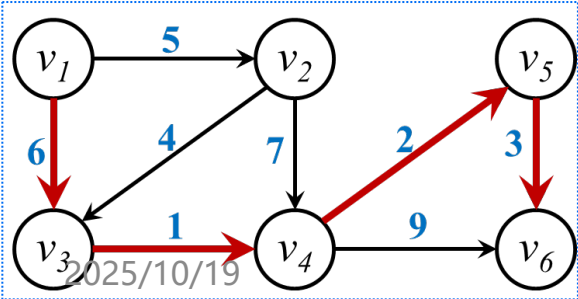
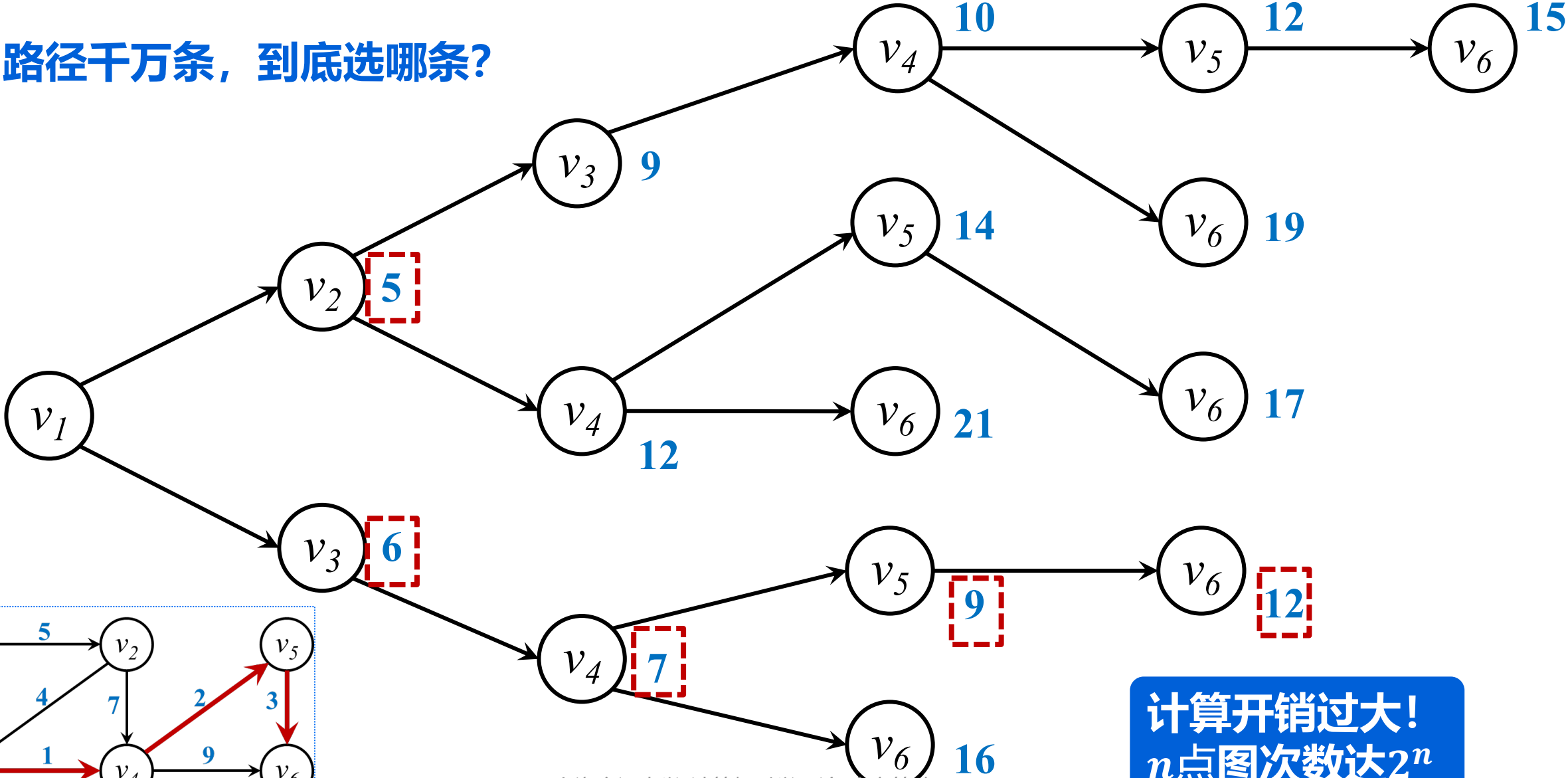


$$d(v_1, v_6) = 12$$

$$d(v_1, v_2) = 5, d(v_1, v_3) = 6, d(v_1, v_4) = 7, d(v_1, v_5) = 9.$$

最短路径问题难度

路径千万条，到底选哪条？



计算开销过大!
 n 点图次数达 2^n

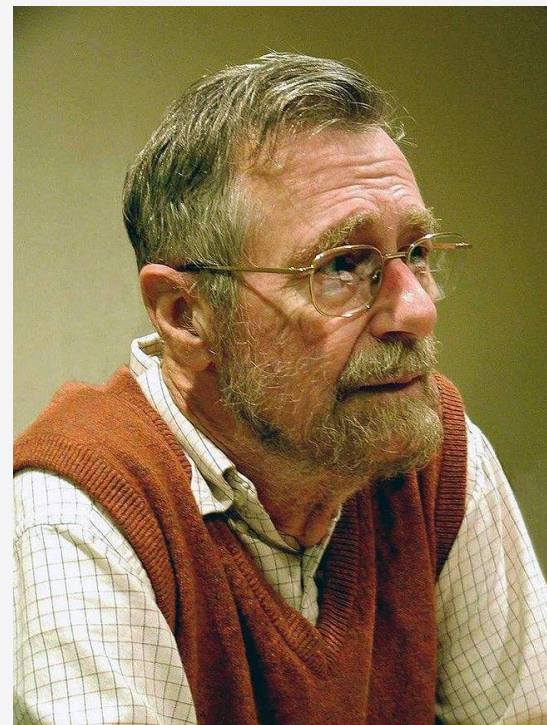
Dijkstra算法由来

Dijkstra算法（贪心算法）

- 数据结构：优先级队列
- 最流行的算法：地图导航、城市规划、管网布局

小故事：“二十分钟影响二十世纪”

- 缘起：1956年ARMAC会议需向大众展示计算机功能
- 灵感：介绍荷兰旅行，需要计算两城市间最短路径
- 简洁：没有纸笔，在咖啡店里**20**分钟想出算法



Edsger Dijkstra

荷兰计算机科学家
1972年获图灵奖

Dijkstra算法步骤与伪代码

- ① 将顶点集合 V 一分为二, S 为已求得最短路径的顶点集合, $Q = V - S$ 为优先级队列
- ② 将 Q 中与源点 s 距离最小的顶点 u 加入集合 S , 更新 u 相邻顶点与 s 的距离
- ③ 重复②, 直至 S 中包含所有顶点

初始化源点 s 的距离为0, 将 s 加入 Q

初始化顶点 v 与 s 的距离为 ∞ ; 将 v 加入 Q

取出 Q 中距离最小顶点

更新 Q 中顶点与源点的距离

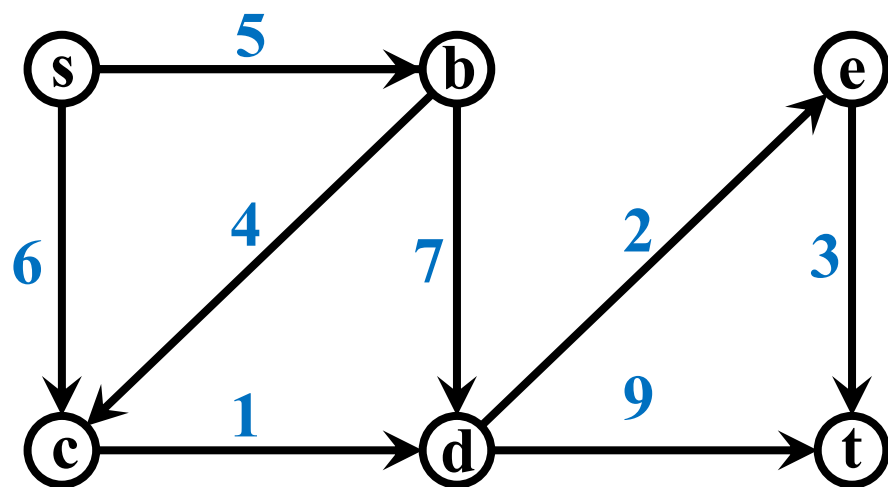
Dijkstra's Algorithm(G, s)

Input: $G = (V, E), s \in V$

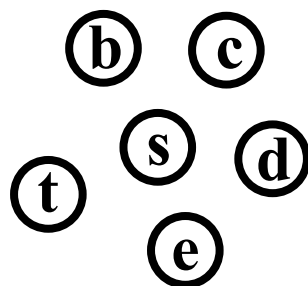
Output: $d[v]$ for all $v \in V$

1. $d[s] = 0$; INSERT(Q, s)
2. **for** each $u \in V, u \neq s$ **do**
3. $d[u] = \infty$; INSERT(Q, u)
4. **While** $Q \neq \emptyset$ **do**
5. $u \leftarrow$ EXTRACT-MIN(Q)
6. $S \leftarrow S \cup \{u\}$
7. **for** each edge $(u, v) \in E$ **do**
8. **if** $d[v] > d[u] + w(u, v)$ **then**
9. $d[v] \leftarrow d[u] + w(u, v)$
10. DECREASE-KEY(Q, v)

Dijkstra算法计算示例：初始状态



集合 V



输入图结构，求s到t最短路径
其中集合V含有图的所有顶点

访问点 →

--

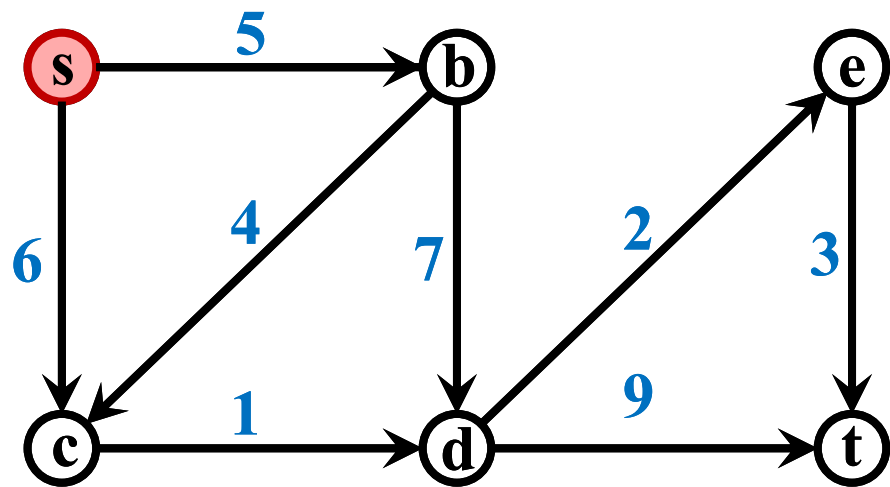
优先队列元素 →

距离标记 →

集合S元素 →

到源点距离 →

Dijkstra算法计算示例：访问s



集合 Q (优先队列)

集合 S



所有点加入优先级队列

取出距离最小的点

加入已知距离集合

访问点 $\rightarrow [s]$

优先队列元素 $\rightarrow$

b	c	d	e	t
∞	∞	∞	∞	∞

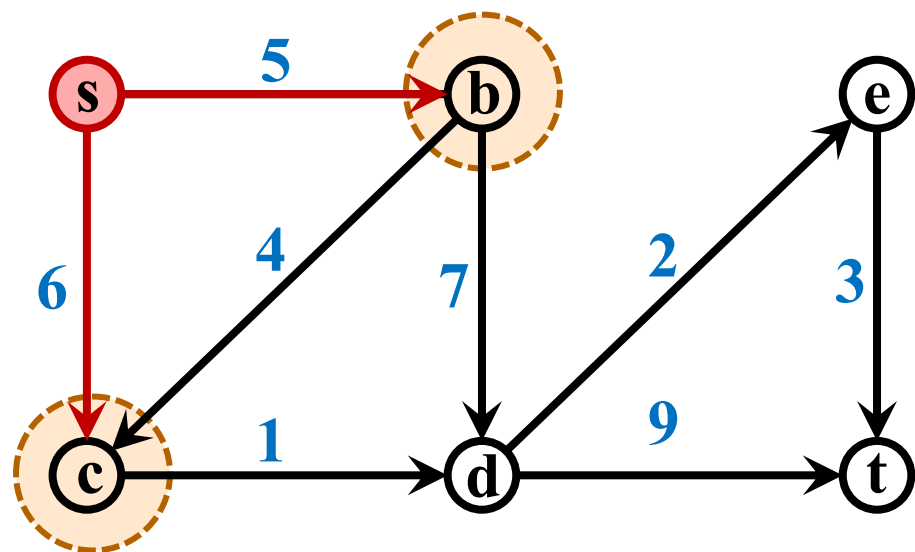
距离标记 $\rightarrow$

集合 S 元素 $\rightarrow$

s					
0					

到源点距离 $\rightarrow$

Dijkstra算法计算示例：访问s



集合 Q (优先队列)

集合 S



访问可达的相邻点

更新邻点距离标记

访问点 $\rightarrow [s]$

优先队列元素 $\rightarrow$

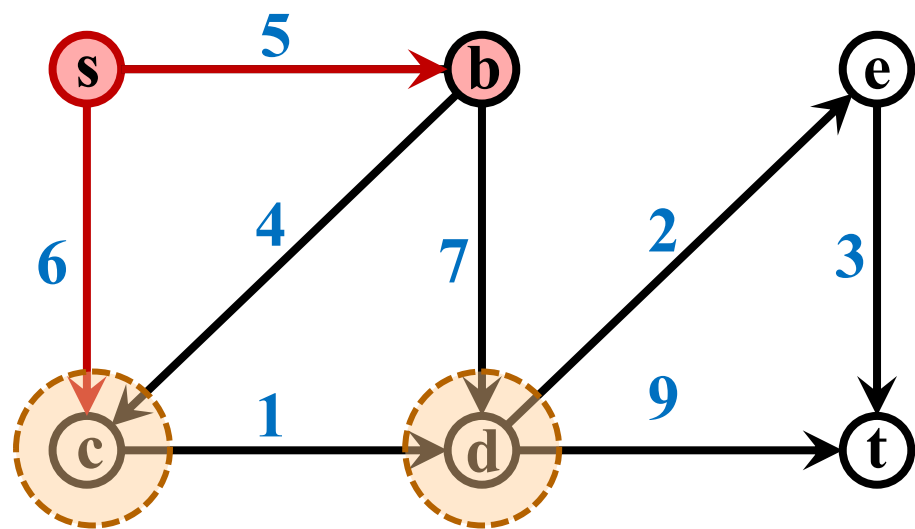
b	c	d	e	t
5	6	∞	∞	∞

距离标记 $\rightarrow$

集合 S 元素 $\rightarrow$

s					
0					

Dijkstra算法计算示例：访问 b



集合 Q (优先队列)

集合 S



取出距离最小的点

更新可达的相邻点

访问点 $\rightarrow [b]$

优先队列元素 $\rightarrow$

c	d	e	t
6	12	∞	∞

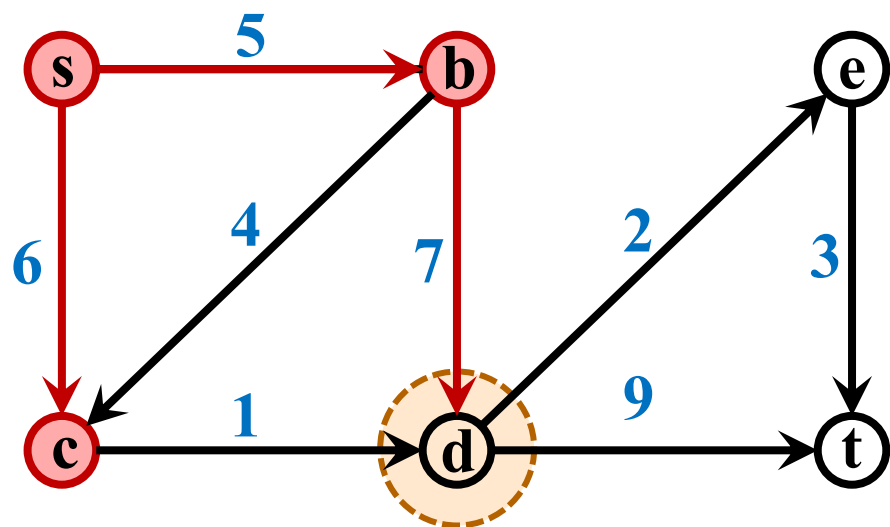
距离标记 $\rightarrow$

集合 S 元素 $\rightarrow$

s	b				
0	5				

到源点距离 $\rightarrow$

Dijkstra算法计算示例：访问c



集合 Q (优先队列)

集合 S



取出距离最小的点

更新可达的相邻点

访问点 $\rightarrow [c]$

优先队列元素 $\rightarrow$

d	e	t
7	∞	∞

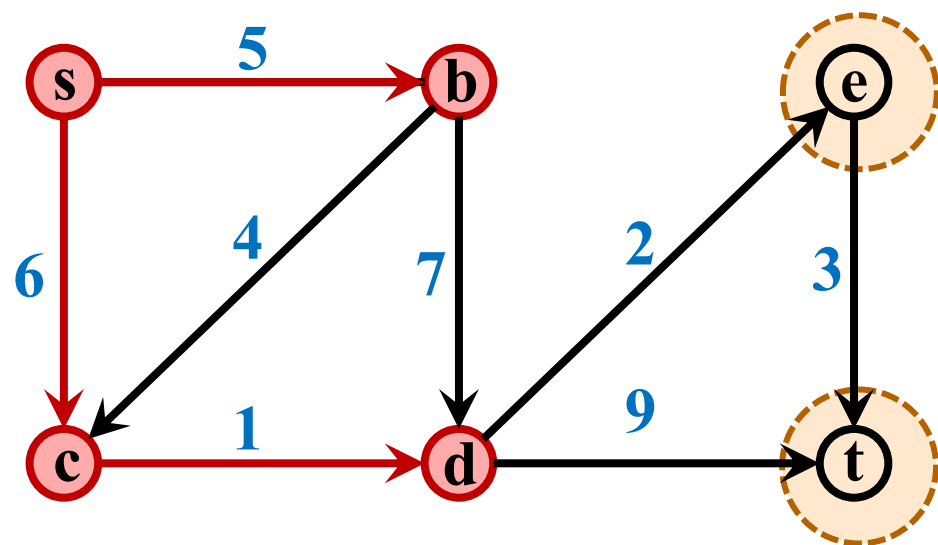
距离标记 $\rightarrow$

集合 S 元素 $\rightarrow$

s	b	c			
0	5	6			

到源点距离 $\rightarrow$

Dijkstra算法计算示例：访问 d



集合 Q (优先队列)

集合 S



取出距离最小的点

更新可达的相邻点

访问点 $\rightarrow [d]$

优先队列元素 $\rightarrow$

e	t
9	16

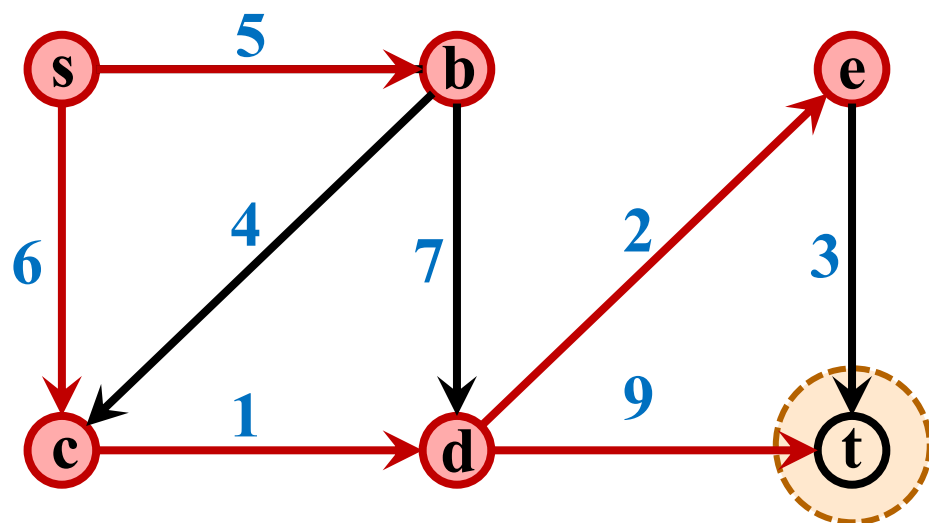
距离标记 $\rightarrow$

集合 S 元素 $\rightarrow$

s	b	c	d		
0	5	6	7		

到源点距离 $\rightarrow$

Dijkstra算法计算示例：访问 e



集合 Q (优先队列)

集合 S

t

b c
 s d
 e

取出距离最小的点

更新可达的相邻点

访问点 $\rightarrow [e]$

优先队列元素 $\rightarrow$

t

距离标记 $\rightarrow$

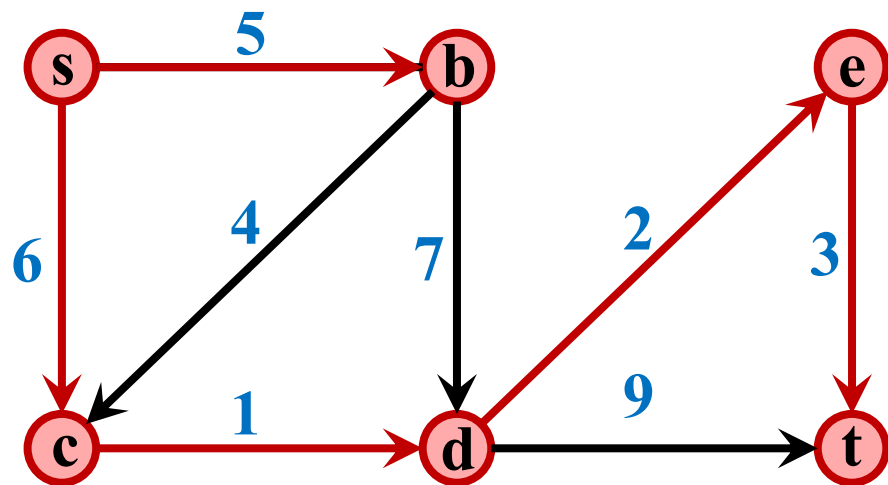
12

集合 S 元素 $\rightarrow$

s	b	c	d	e	
0	5	6	7	9	

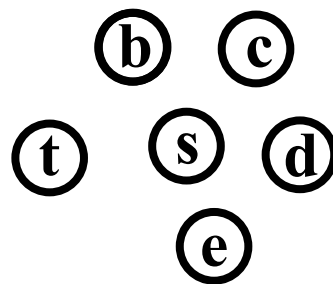
到源点距离 $\rightarrow$

Dijkstra算法计算示例：访问 t



集合 Q (优先队列)

集合 S



取出距离最小的点

更新可达的相邻点

访问点 $\rightarrow$ $[t]$

找到最短路
算法结束

优先队列元素 $\rightarrow$

距离标记 $\rightarrow$

集合 S 元素 $\rightarrow$

到源点距离 $\rightarrow$

目录

01

贪心算法简介

02

最短路算法

03

贪心算法应用

贪心算法的应用

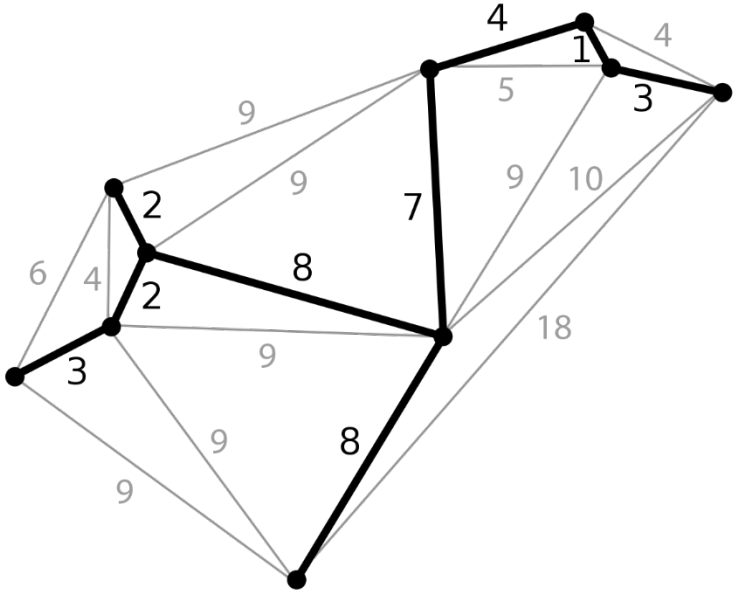
寻路算法

- 单源最短路径
- 带权图
- 地图寻址、最优路线



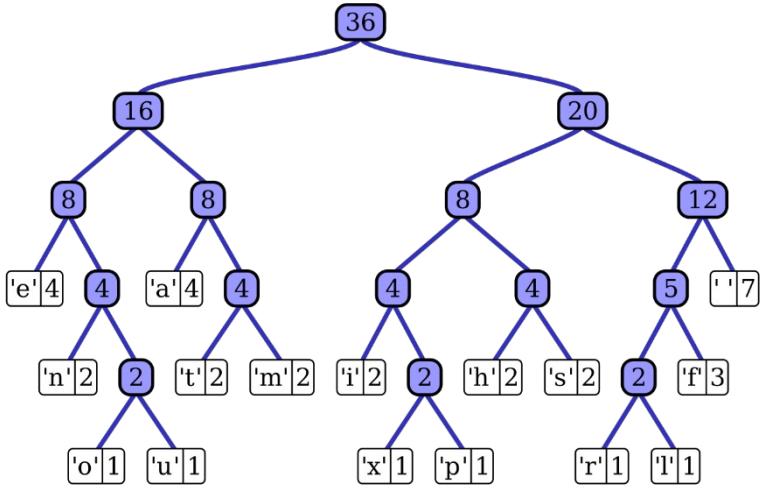
图探索算法

- 最小生成树
- 带权图
- 管道铺设、最小开销



哈夫曼编码算法

- 将字母转化为不等长二进制字符串
- 降低存储量
- 信道传输、文件存储



电影《华尔街》



贪心算法

- 背包问题：贪心算法快速，但不一定正确
- 工厂生产问题：贪心算法正确性与贪心策略有关
- 硬币找零问题：贪心算法正确性与问题设定有关

最短路算法

- Dijkstra算法

贪心算法的应用

- 寻路算法
- 图探索算法
- 哈夫曼编码算法



谢谢!

```
#modifier_ob
modifier_ob = bpy.context.selected_objects[0]
print("Modifier object:" +str(modifier_ob.name))

#modifier_ob.select=1

print("mirror_ob",mirror_ob)
print("modifier_ob",modifier_ob)

# put mirror modifier on modifier_ob

mirror_mod = modifier_ob.modifiers.new("mirror_mirror","MIRROR")

# set mirror object to mirror_ob
mirror_mod.mirror_object = mirror_ob

if _operation == "MIRROR_X":
    mirror_mod.use_x = True
    mirror_mod.use_y = False
    mirror_mod.use_z = False
```

